

parthkotwal / olds

Q

Type / to search

Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

olds

Public

Unpin

Watch 0

Fork 0

Star 0

About

readolds.fyi

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Deployments 30

olds / production

Production 2 months ago

spirited-gentleness / production

Packages

No packages published

Publish your first package

Contributors 1

parthkotwal

Parth Kotwal

Languages

Go 61%

JavaScript 25.2%

Python 10.4%

Dockerfile 1.8%

Other 1.6%

1 Branch

0 Tags

Go to file

Go to file

Add file

<> Code

 **parthkotwal** read me 

64199cb · 2 months ago

 36 Commits

 .claude	frontend redesign	2 months ago
 .github/workflows	Add CI workflow, expose metrics e...	2 months ago
 backend	Add JSON message for explanatio...	2 months ago
 docs	fixed file	5 months ago
 frontend	fix explanations not showing up	2 months ago
 ml-service	redeploying!	2 months ago
 .env.example	LLM added for explanations + spor...	5 months ago
 .gitignore	frontend redesign	2 months ago
 README.md	read me	2 months ago
 docker-compose.yml	redeploying!	2 months ago

README  

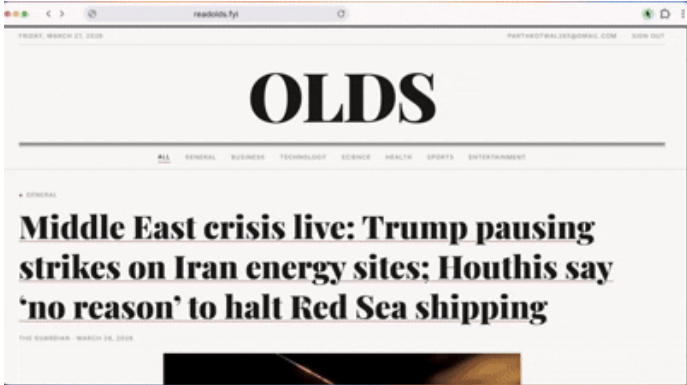
Olds

A news reader that show you non-obvious connections between stories across topics.

 CI

passing

 readolds.fyi

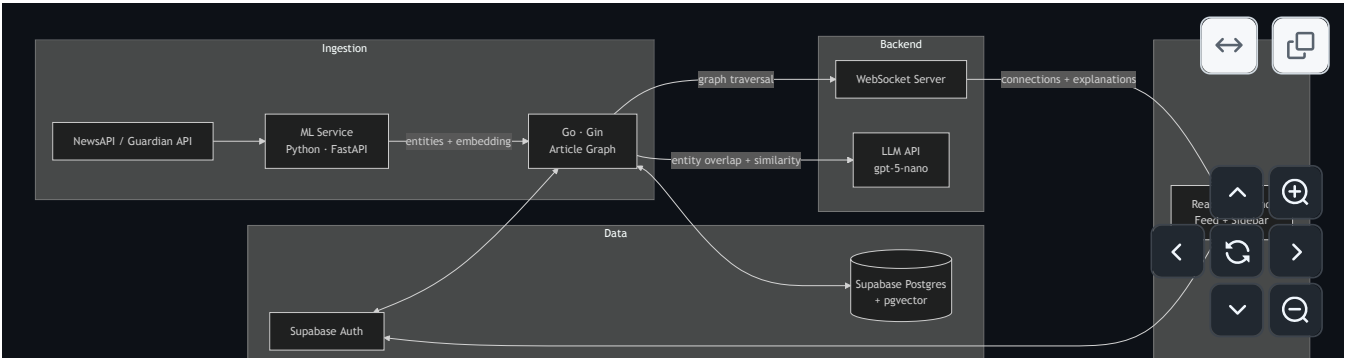


Olds finds connections between news stories that don't belong to the same category. When you read an article about Iranian trade sanctions, the sidebar shows you a related crime story in Jakarta involving the same political actors — and explains *why* they're connected in plain English. The feed learns from how you read, not what you explicitly rate.

How it works

Articles arrive from NewsAPI and The Guardian every 30 minutes. A Python service extracts named entities (people, places, organizations) and generates a 384-dimensional sentence-transformer embedding for each article. The Go backend stores both and builds an in-memory directed graph where edge weights combine cosine similarity of embeddings with named entity overlap, so two articles are connected both semantically *and* concretely.

When you open an article, Go traverses the graph in real time, filters for cross-topic neighbors above a similarity threshold, and pushes them to the browser over a WebSocket. An LLM then generates a 1–2 sentence natural language explanation for each connection, streaming results into the sidebar as they resolve. The feed ranking is shaped by implicit signals — dwell time, scroll depth, re-opens — with no explicit ratings.



Metrics

Metric	Value
Articles indexed	11,464
Graph edges	953,251 unique edges
Cross-topic connection rate	59.0%
Enrichment success rate	83.5%
Graph traversal latency	p50 382 ms / p95 819 ms / p99 819 ms
WebSocket push latency	p50 0 ms / p95 0 ms / p99 0 ms
ML inference per article	p50 699 ms / p95 2,795 ms / p99 5,606 ms

Runtime metrics are exposed through `GET /metrics`, including article count, graph edge count, cross-topic ratio, latency percentiles, ingestion counts, and LLM explanation cache size. Detailed historical snapshots remain behind authenticated `/stats` endpoints.

Local development

```
git clone https://github.com/parthkotwal/olds.git
cd olds
cp .env.example .env
# Fill in: SUPABASE_DATABASE_URL, SUPABASE_JWT_SECRET, SUPABASE_ANON_KEY,
#          SUPABASE_URL, NEWSAPI_KEY, GUARDIAN_API_KEY, LLM_API_KEY
docker compose up
```



Visit localhost:3000. The backend runs on `:8080`, ML service on `:8001`.

On first start the backend triggers an ingestion run. It takes 2–3 minutes to fetch articles, run them through the ML service, and build the graph. The feed populates automatically; no manual step needed.

Courtesy of Parth Kotwal